

RÉPUBLIQUE DÉMOCRATIQUE DU CONGO
ENSEIGNEMENT SUPÉRIEUR ET UNIVERSITAIRE
UNIVERSITÉ CATHOLIQUE DE BUKAVU
(U.C.B)
BP. 285 BUKAVU



FACULTÉ DES SCIENCES ET TECHNOLOGIES
Département des Sciences de l'Informatique

DÉTECTION D'ANOMALIES SUR CAPTEURS IoT AVEC UN MODÈLE EDGE AI

Présenté par le groupe N° 9 constitué de :

ANSIMA BISIMWA Stéphane
IMANI BAHATI Abraham
BUTEGHEKANE KITATUTA Yseult
MUNGELA ADASA Esther
MPOYI MUAMBA Mike
KATABANA ASHUZA Sandra
NSIMIRE MANGO Douceline
MUSAFIRI Jean-Damas Charmant

Encadreurs :

Prof. Jules KALA
Ass. BIRINDWA Joviane

Année académique 2025–2026

Résumé

Ce travail porte sur la détection d'anomalies dans des mesures de capteurs IoT au moyen d'un modèle d'Edge AI. Le jeu de données utilisé provient du référentiel UCI *Occupancy Detection* et contient des mesures de température, humidité, luminosité, concentration de CO₂, ratio d'humidité et présence. Comme ce jeu de données ne possède pas de colonne indiquant explicitement si une observation est *normale* ou *anormale*, le problème a été reformulé en apprentissage non supervisé. Le modèle apprend le comportement de référence à partir de quatre variables directement interprétables et saisissables par l'utilisateur : **Temperature**, **Humidity**, **Light** et **CO₂**. Des anomalies contrôlées sont injectées uniquement dans les ensembles de validation et de test pour permettre une évaluation mesurable.

Le modèle retenu est un autoencodeur compact de structure 4–16–3–16–4, totalisant 263 paramètres entraînés. Après normalisation par **RobustScaler**, l'autoencodeur reconstruit les observations normales et produit un score d'anomalie à partir de l'erreur quadratique moyenne. Le seuil de décision sélectionné sur l'ensemble de validation est de 0,015398. Sur le test final, le système obtient 80,87 % d'accuracy, 74,43 % de précision, 94,06 % de rappel, 83,10 % de F1-score, 93,05 % de ROC-AUC, ainsi qu'un MAE de 0,1145, un RMSE de 0,1673 et un R^2 de 0,8718 pour la reconstruction des observations normales.

Le projet a connu deux étapes d'interface. Un premier prototype a été développé sous Streamlit pour valider rapidement la chaîne de prédiction. Ensuite, afin de mieux représenter le principe Edge AI, ce prototype a été remplacé par une Progressive Web App nommée **EdgePulse**. Dans cette version, les poids du modèle, les paramètres de normalisation et la règle de décision sont exécutés localement dans le navigateur. L'application peut être ouverte depuis le Web, installée comme PWA et utilisée hors ligne après mise en cache. En l'absence de capteurs physiques et de carte embarquée disponible, la démonstration repose sur la saisie manuelle des valeurs, mais conserve une logique fidèle à une exécution au plus près du capteur.

Mots-clés : IoT, Edge AI, autoencodeur, détection d'anomalies, PWA, inférence locale.

Table des matières

1	Introduction	4
2	Données et méthodologie	4
2.1	Origine des données	4
2.2	Choix des variables	5
2.3	Formulation de l'anomalie	5
3	Conception du modèle	5
3.1	Architecture retenue	5
3.2	Prétraitement et apprentissage	6
3.3	Choix du seuil	6
4	Résultats expérimentaux	7
5	Premier prototype sous Streamlit	7
6	Passage à la PWA EdgePulse	9
7	Interface finale de l'application	10
8	Adaptation mobile, discussion et conclusion	11
8.1	Maquettes mobiles de la PWA	11
8.2	Discussion	11
8.3	Conclusion	11
	Références	13

Table des figures

1	Évolution de la perte d'entraînement et de validation.	7
2	Évaluation graphique du modèle.	8
3	Premières vues de l'interface Streamlit.	8
4	Exemples de résultats produits par le prototype Streamlit.	9
5	Vues principales de l'interface Web EdgePulse.	10
6	Autres écrans de l'application EdgePulse.	10
7	Exemples de rendu mobile de la PWA EdgePulse.	11
8	Accès direct à l'application et au code source.	12

1. Introduction

L’Internet des objets (IoT) permet de relier des dispositifs physiques capables de mesurer leur environnement et de transmettre ces mesures à un système de traitement. Dans un local technique, une salle de cours, un bureau ou un environnement domestique, des variables telles que la température, l’humidité, la luminosité ou le CO₂ peuvent révéler des situations inhabituelles : dérive thermique, mauvaise ventilation, occupation inattendue ou comportement anormal du système surveillé.

Dans une architecture classique, les données remontent vers un serveur distant où l’analyse est réalisée. Cette solution est pratique, mais elle dépend du réseau, introduit une latence et suppose le transfert permanent des données brutes. À l’inverse, l’Edge AI rapproche le calcul du lieu de mesure. Le modèle est exécuté localement sur un dispositif proche du capteur, ce qui réduit la latence, limite la dépendance à la connexion et améliore la confidentialité.

Dans le cadre de ce travail, nous ne disposons pas de capteurs physiques ni d’une carte embarquée de type Raspberry Pi ou ESP32. L’objectif a donc été de construire un **prototype crédible d’inférence Edge**, dans lequel la source des valeurs est simulée par saisie manuelle, tandis que le prétraitement, le modèle et la décision restent exécutés localement.

Objectif général. Concevoir un modèle compact de détection d’anomalies à partir de données IoT réelles, puis le rendre exploitable dans une application installable et utilisable sur téléphone, même sans connexion après chargement initial.

2. Données et méthodologie

2.1 Origine des données

Le jeu de données fourni par l’enseignant correspond au jeu UCI *Occupancy Detection*. Il a été initialement conçu pour prédire l’occupation d’une pièce à partir de mesures ambiantes. Dans ce contexte, la variable *Occupancy* vaut 0 ou 1 selon qu’une personne est présente ou non dans la pièce.

Table 1 – Découpage des fichiers utilisés

Fichier	Nombre de lignes	Présence = 1	Rôle dans ce travail
datatraining.txt	8 143	1 729	apprentissage du comportement de référence
datatest.txt	2 665	972	validation et choix du seuil
datatest2.txt	9 752	2 049	évaluation finale

2.2 Choix des variables

Les variables retenues dans le modèle sont `Temperature`, `Humidity`, `Light` et `CO2`. `HumidityRatio` a été écartée parce qu'elle est dérivée principalement de la température et de l'humidité, et `Occupancy` n'a pas été utilisée comme cible, car elle n'indique pas une anomalie.

Table 2 – Paramètres de normalisation exportés avec le modèle

Variable	Médiane d'entraînement	Échelle robuste
Temperature	20,39	1,69
Humidity	26,2225	10,3333
Light	0	256,375
CO ₂	453,5	199,8333

2.3 Formulation de l'anomalie

L'absence de colonne `Anomaly` nous a conduits à adopter une démarche non supervisée. Les données d'entraînement sont laissées intactes. Des anomalies contrôlées sont générées uniquement pour la validation et le test par perturbation d'une ou deux variables. Les amplitudes de perturbation sont résumées dans le tableau suivant.

Table 3 – Perturbations utilisées pour créer des anomalies contrôlées

Variable	Règle de perturbation
Température	ajout ou retrait aléatoire de 3 à 7 °C
Humidité	ajout ou retrait de 12 à 30 points
Lumière	ajout de 400 à 1 200 lux
CO ₂	ajout de 700 à 2 500 ppm

Dans environ 70 % des cas, une seule variable est perturbée ; dans 30 % des cas, deux variables le sont simultanément. Ce protocole permet d'évaluer la sensibilité du détecteur, tout en restant transparent sur le caractère semi-synthétique de l'évaluation.

3. Conception du modèle

3.1 Architecture retenue

Nous avons choisi un autoencodeur dense, volontairement compact, afin de rester cohérents avec une logique d'Edge AI. Son architecture est la suivante :

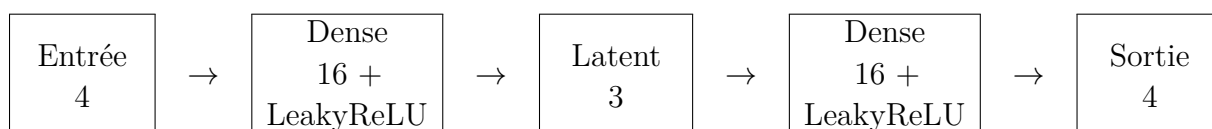


Table 4 – Répartition des paramètres entraînés

Couche	Entrée	Sortie	Paramètres
Dense 1 (encodeur)	4	16	80
Dense 2 (encodeur)	16	3	51
Dense 1 (décodeur)	3	16	64
Dense 2 (décodeur)	16	4	68
Total			263

3.2 Prétraitement et apprentissage

Chaque entrée est normalisée à l’aide d’un `RobustScaler` selon la formule

$$x' = \frac{x - \text{médiane}}{\text{échelle robuste}}.$$

Ce choix limite l’influence des valeurs extrêmes. Les mêmes paramètres sont réutilisés côté navigateur dans la PWA.

Le modèle a été entraîné sous PyTorch avec l’optimiseur Adam, un taux d’apprentissage de 10^{-3} , une taille de lot de 128 et un maximum de 150 époques. Un *early stopping* de patience 20 a été utilisé pour retenir le meilleur état du modèle. La perte d’entraînement est l’erreur quadratique moyenne (MSE) entre l’entrée normalisée et sa reconstruction.

Table 5 – Principaux hyperparamètres

Élément	Valeur
Optimiseur	Adam
Taux d’apprentissage	0,001
Taille de lot	128
Nombre maximal d’époques	150
Patience de l’early stopping	20
Époque retenue	146
MSE d’entraînement finale	0,00249

3.3 Choix du seuil

Après l’entraînement, les scores de reconstruction sont calculés sur l’ensemble de validation. Le seuil est choisi par balayage de valeurs candidates en retenant celui qui maximise le F1-score. Le seuil retenu est **0,015398**. La règle de décision devient donc :

$$\text{Anomalie si } \text{MSE}(x, \hat{x}) > 0,015398.$$

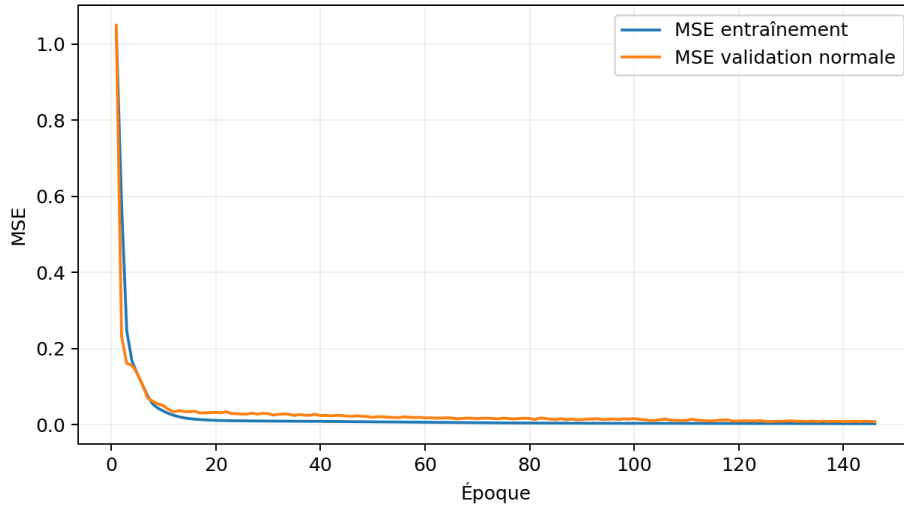


Figure 1 – Évolution de la perte d'entraînement et de validation.

4. Résultats expérimentaux

Le test final combine les 9 752 observations de référence de `datatest2.txt` et 9 752 anomalies contrôlées construites à partir de ces mêmes observations. Les métriques de reconstruction ne doivent pas être confondues avec les métriques de détection.

Table 6 – Métriques principales de détection

Accuracy	Precision	Recall	F1-score	ROC-AUC
80,87 %	74,43 %	94,06 %	83,10 %	93,05 %

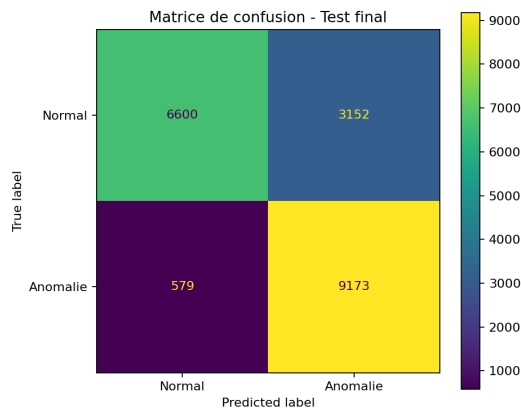
Table 7 – Métriques de reconstruction sur les observations normales

MAE	RMSE	R^2
0,1145	0,1673	0,8718

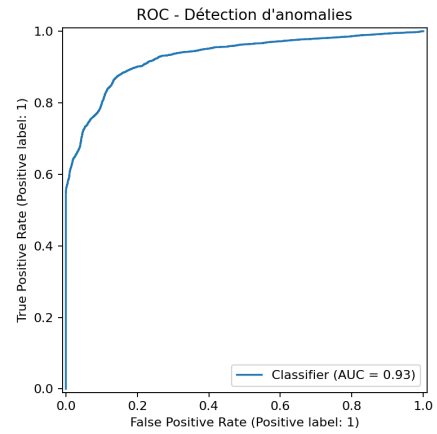
La matrice de confusion correspond à 6 600 vrais négatifs, 3 152 faux positifs, 579 faux négatifs et 9 173 vrais positifs. Le rappel élevé signifie que la majorité des anomalies contrôlées est détectée. En revanche, le taux de faux positifs reste important : 32,32 % des observations de référence du test sont classées comme anomalies. Ce résultat est acceptable pour un prototype académique, mais il indique qu'un recalibrage serait nécessaire avant une utilisation opérationnelle.

5. Premier prototype sous Streamlit

Une première interface a été développée sous Streamlit pour vérifier la chaîne de bout en bout : saisie des valeurs, chargement du modèle, calcul du score et affichage du résultat.



(a) Matrice de confusion.



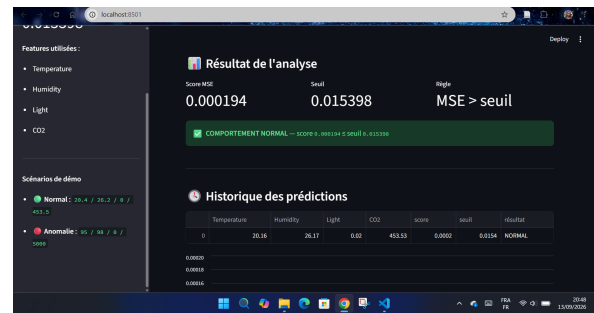
(b) Courbe ROC.

Figure 2 – Évaluation graphique du modèle.

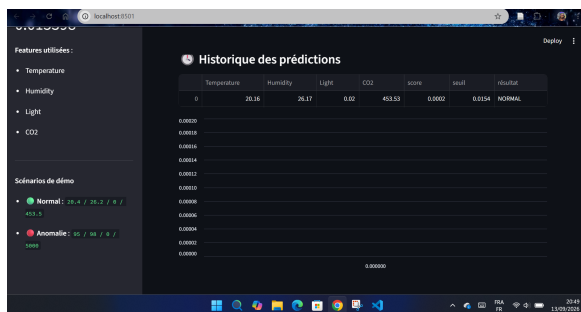
Les captures suivantes sont conservées parce qu'elles témoignent de la première étape du projet.



(a) Dashboard avec valeurs par défaut.



(b) Résultat normal.



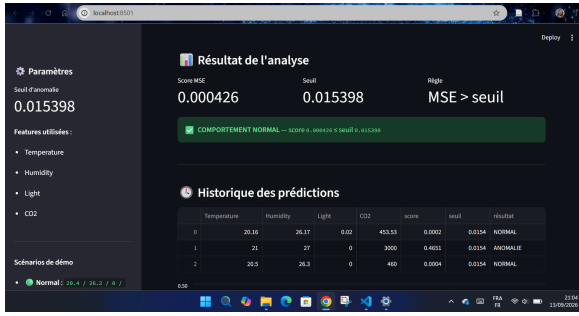
(c) Historique des prédictions.



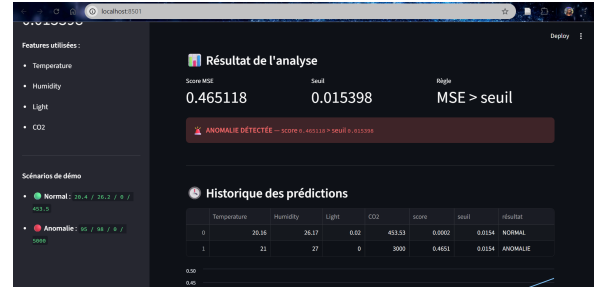
(d) Insertion d'un autre jeu de valeurs.

Figure 3 – Premières vues de l'interface Streamlit.

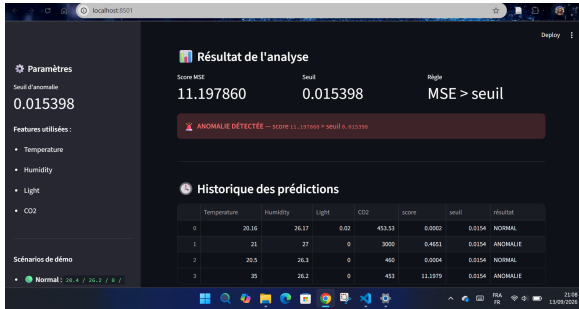
Ce prototype nous a aidés à vérifier la cohérence du modèle, mais il restait lié à un environnement Python. Dans notre contexte, il ne répondait pas totalement à l'idée d'une exécution *sur l'appareil de l'utilisateur*, ni à la possibilité d'installation comme application mobile.



(a) Résultat de l'analyse.



(b) Premier cas d'anomalie.



(c) Deuxième cas d'anomalie.



(d) Historique et graphique récapitulatifs.

Figure 4 – Exemples de résultats produits par le prototype Streamlit.

6. Passage à la PWA EdgePulse

Pour mieux représenter le paradigme Edge AI, l'interface finale a été repensée sous forme de Progressive Web App nommée **EdgePulse**. Ce choix a été motivé par plusieurs besoins :

- disposer d'une application accessible directement par une URL ;
- permettre l'installation sur téléphone comme une application légère ;
- faire fonctionner l'interface hors connexion après mise en cache ;
- exécuter la logique du modèle dans le navigateur, sans requête serveur pour chaque prédiction.

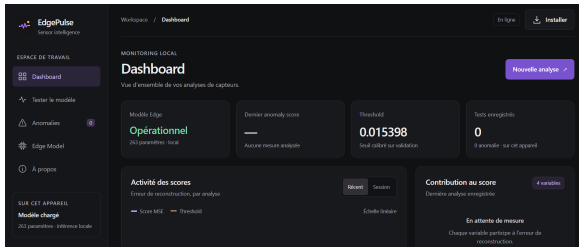
Principe de fonctionnement dans la PWA. L'utilisateur saisit les quatre valeurs, le navigateur applique la même normalisation que la version Python, reconstruit l'observation au moyen des poids exportés, calcule l'erreur MSE et compare cette erreur au seuil 0,015398.



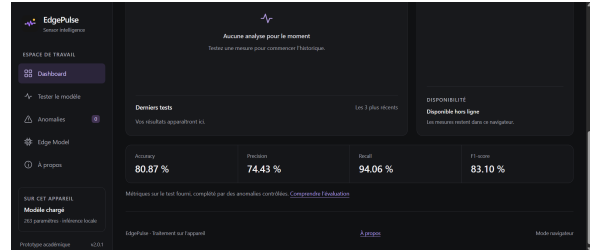
L'absence de capteurs physiques reste une limite importante. Le téléphone utilisé pour la démonstration ne mesure pas directement la lumière, l'humidité ou le CO₂ dans les conditions du projet. La PWA doit donc être comprise comme un **banc de test d'inférence Edge** et non comme un système complet d'acquisition matérielle.

7. Interface finale de l'application

Les captures ci-dessous correspondent à l'application actuellement disponible à l'adresse anomaly-detection-azure.vercel.app/dashboard. Elles montrent le tableau de bord, l'écran de test, la page des anomalies, la fiche du modèle et la page de présentation.



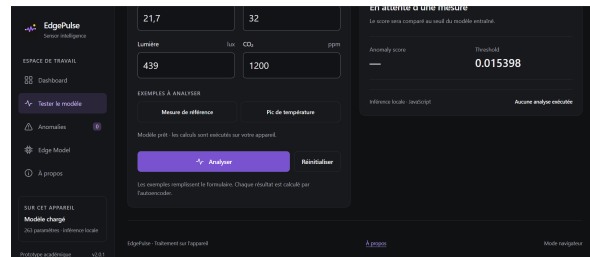
(a) Début du tableau de bord.



(b) Suite du tableau de bord.

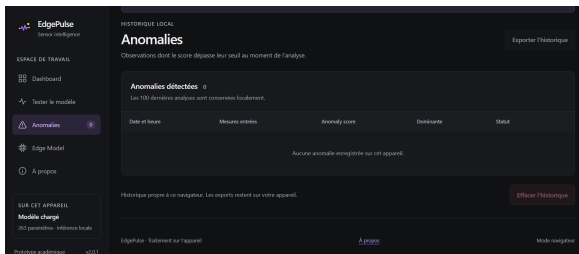


(c) Écran « Tester le modèle ».

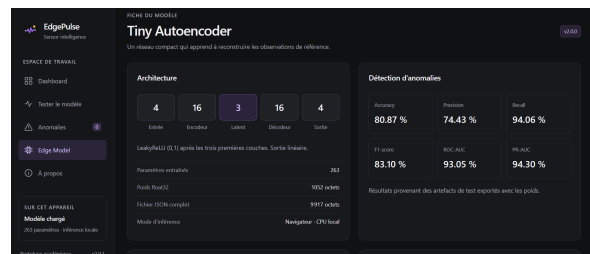


(d) Zone d'action et d'analyse.

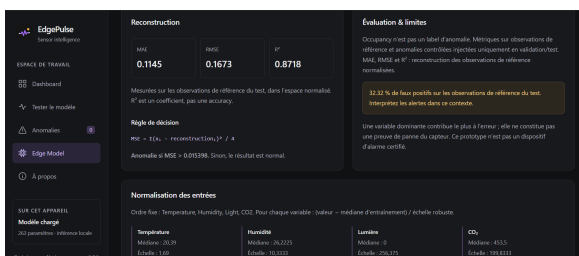
Figure 5 – Vues principales de l'interface Web EdgePulse.



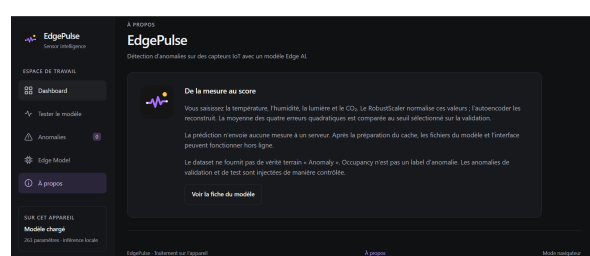
(a) Historique des anomalies.



(b) Fiche du modèle : architecture et métriques.



(c) Règle de décision et normalisation.



(d) Page « À propos ».

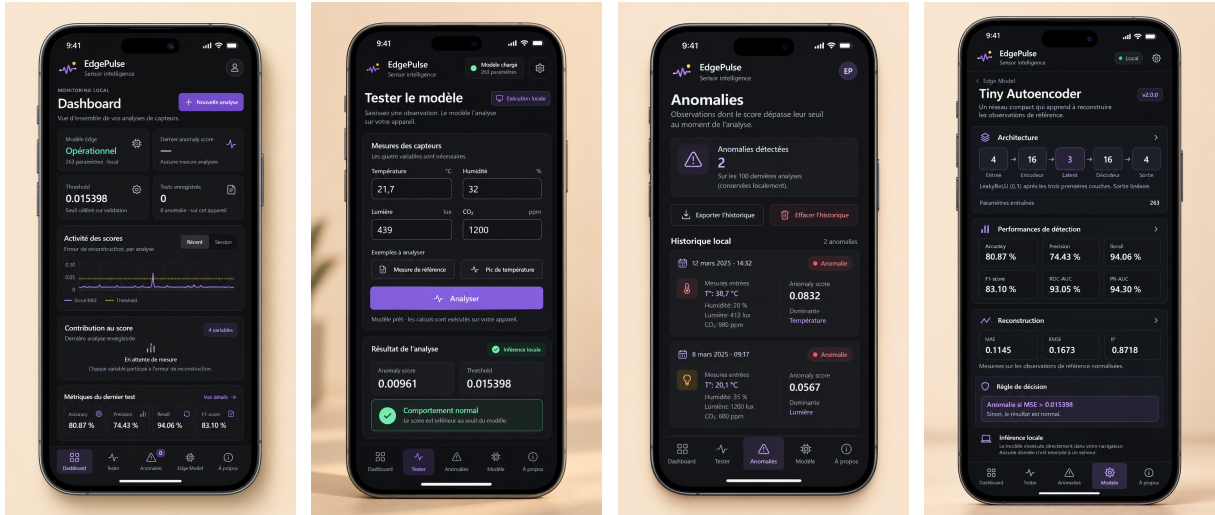
Figure 6 – Autres écrans de l'application EdgePulse.

L'application présente de manière lisible le seuil, l'architecture du modèle, les métriques, l'historique local des analyses et une estimation de la contribution des variables au score de reconstruction. Cette dernière indication a une valeur interprétative, mais ne doit pas être interprétée comme un diagnostic physique définitif de panne.

8. Adaptation mobile, discussion et conclusion

8.1 Maquettes mobiles de la PWA

Les maquettes suivantes illustrent le rendu visé sur smartphone. Elles complètent les captures du navigateur et aident à documenter la dimension mobile du projet.



(a) Dashboard mobile. (b) Test du modèle sur (c) Historique des anomalies. (d) Fiche du modèle.
smartphone.

Figure 7 – Exemples de rendu mobile de la PWA EdgePulse.

8.2 Discussion

Le travail présente plusieurs points forts. D’abord, la chaîne est complète : données, prétraitement, apprentissage, validation, test, export des poids et intégration dans une application installable. Ensuite, le modèle est extrêmement compact, avec seulement 263 paramètres. Enfin, le passage de Streamlit à la PWA rend la démonstration plus fidèle à une logique Edge AI.

Cependant, certaines limites doivent être rappelées. Le dataset d’origine ne contient pas de vérité terrain « anomalie », et l’évaluation repose donc sur des anomalies contrôlées. Le taux de faux positifs reste élevé. De plus, l’application ne collecte pas directement les mesures depuis des capteurs physiques. Une validation future sur Raspberry Pi, microcontrôleur ou chaîne MQTT serait nécessaire pour parler d’un système déployé.

8.3 Conclusion

Ce projet a permis de montrer qu’il est possible de concevoir un détecteur d’anomalies léger, de l’entraîner à partir de données IoT existantes, puis de l’intégrer dans une application Web installable. Le premier prototype Streamlit a servi à valider rapidement la logique. La version finale **EdgePulse** améliore la démonstration en rapprochant effectivement l’inférence de l’utilisateur final. Malgré les limites liées à l’absence de capteurs réels et à

la nature semi-synthétique de l'évaluation, le résultat constitue une preuve de concept cohérente d'Edge AI pour l'analyse locale de données IoT.



Lien de l'application PWA

[anomaly-detection-
azure.vercel.app/dashboard](https://anomaly-detection-azure.vercel.app/dashboard)



Lien du dépôt GitHub

github.com/AbraImani/Anomaly_Detection

Figure 8 – Accès direct à l'application et au code source.

Références

- [1] L. Candanedo, *Occupancy Detection*, UCI Machine Learning Repository, 2016.
- [2] L. M. Candanedo and V. Feldheim, “Accurate occupancy detection of an office room from light, temperature, humidity and CO₂ measurements using statistical learning models,” *Energy and Buildings*, vol. 112, pp. 28–39, 2016.
- [3] I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
- [4] C. Zhou and R. C. Paffenroth, “Anomaly Detection with Robust Deep Autoencoders,” *Proceedings of KDD*, 2017.
- [5] W. Shi et al., “Edge Computing : Vision and Challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [6] MDN Web Docs, *Progressive Web Apps*. <https://developer.mozilla.org/>.